第二十章　推理加速与成本控制

20.1　KV Cache：为什么生成第 100 个词不需要重新算前 99 个词

20.1.1　重复了多少次

大模型生成文本是一个 token 一个 token 往外蹦的。生成第 1 个 token，拼到输入后面，再生成第 2 个，再拼上去，再生成第 3 个……这个过程前面已经反复提过了。但有一个问题你可能从来没有仔细算过：在这个“逐个生成”的过程中，模型总共处理了多少次 token？

假设模型要生成一段 200 个 token 的回答。生成第 1 个 token 的时候，模型要处理用户输入的那些 token。生成第 2 个的时候，要把之前所有的 token 加上新的，重新处理一遍。生成第 3 个的时候，再全部重新处理一遍。以此类推，每多生成一个 token，之前所有 token 都要再过一遍模型。

我们只看“已生成的 token 被重复处理了多少次”。第 1 个生成的 token，在后续的 199 步里每步都会被重新处理一次；第 2 个在后续 198 步里被重新处理；第 3 个在后续 197 步里被重新处理。加起来，光是重复处理就有 $199+198+\cdots+1=19900$ 次。

接近两万次。绝大多数都是重复的。第 1 个 token 每次被重新处理，得到的结果都和上一次一模一样——但模型还是老老实实地重新算了一遍。

如果每个 token 只算一次、结果存下来以后直接用呢？那就只有 200 次。

从两万次到两百次，差了一百倍。序列越长，浪费越严重。

这一节要讲的就是怎么把这些重复计算省掉。方案叫 **KV Cache**，名字里的“KV”指的是注意力机制中的 Key 和 Value。为什么偏偏存 Key 和 Value、不存别的？这个问题你不需要死记答案，跟着我走一遍具体的数字，重复发生在哪里、哪些东西能存下来复用，会自己浮出来。

20.1.2　不用缓存，从头算一遍

为了把每一步的计算看得清清楚楚，我们构造一个最小的例子。这个例子会走两轮生成。

第一轮：模型已经看到了 2 个 token，要生成序列中的第 3 个 token。我们先不急着把完整注意力过程全部展开，只盯住一件事：这 2 个 token 的 Key 和 Value 是怎么算出来的。

第二轮：第 3 个 token 已经生成出来，模型接着要生成第 4 个 token。到了这一轮，前 2 个 token 的 Key 和 Value 会被再次用到，重复计算也会暴露出来。那时我们再把完整的注意力过程走一遍。

设定：1 层 Transformer，向量维度是 2 维。模型已经收到了用户输入的 2 个 token，现在要开始生成回答。

两个输入 token 的 Embedding 向量：

Token	Embedding
第 1 个 token	$[1, 0]$
第 2 个 token	$[0, 1]$

注意力机制需要三个矩阵 W_Q、W_K、W_V，把每个 token 的向量分别变换成 Query、Key、Value 三个新向量。我们给这三个矩阵赋上具体的值（数字是我编的，为了让计算过程清晰）：

$$W_Q = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad W_K = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \qquad W_V = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

模型要生成第 3 个 token。为此，它需要把这 2 个 token 送进注意力层，算出最后一个位置的输出向量（最后一个位置的输出决定下一个 token 是什么）。

这一轮当然也会计算 Query、注意力分数和输出向量，但这些细节先按下不表。我们现在只盯一件事：前两个 token 的 Key 和 Value 是怎么被算出来的。等模型生成出第 3 个 token、准备继续生成第 4 个 token 时，重复计算就会暴露出来。

计算的第一步是：把每个 token 的 Embedding 分别乘以 W_K 和 W_V，算出每个 token 的 Key 和 Value。

第 1 个 token $[1, 0]$ 的 Key：

$$[1, 0] \times W_K = [1 \times 2 + 0 \times 0,\ 1 \times 0 + 0 \times 1] = [2, 0]$$

第 1 个 token $[1, 0]$ 的 Value：

$$[1, 0] \times W_V = [1 \times 1 + 0 \times 1,\ 1 \times 1 + 0 \times 0] = [1, 1]$$

第 2 个 token $[0, 1]$ 的 Key：

$$[0, 1] \times W_K = [0 \times 2 + 1 \times 0,\ 0 \times 0 + 1 \times 1] = [0, 1]$$

第 2 个 token $[0, 1]$ 的 Value：

$$[0, 1] \times W_V = [0 \times 1 + 1 \times 1,\ 0 \times 1 + 1 \times 0] = [1, 0]$$

把结果列成一张表：

Token	Embedding	Key	Value
第 1 个 token	$[1, 0]$	$[2, 0]$	$[1, 1]$
第 2 个 token	$[0, 1]$	$[0, 1]$	$[1, 0]$

模型用这些 K 和 V 继续完成注意力计算，算出最后一个位置（也就是第 2 个位置）的输出向量。前面讲过，Decoder 模型总是用序列最后一个位置的输出来预测下一个 token，所以这里就是从第 2 个位置的输出中预测出第 3 个 token。

到这里，第一轮结束。第 3 个 token 具体是什么词不重要；为了继续手算，我们只假设它已经被采样出来了，Embedding 向量是 $[1, 1]$。

现在模型要进入第二轮：生成第 4 个 token。序列已经变成 3 个 token：第 1 个 $[1,0]$，第 2 个 $[0,1]$，第 3 个 $[1,1]$。

不用缓存的做法是：把这 3 个 token 从头到尾全部重新算一遍 Key 和 Value。3 个 token 各自乘以 W_K 和 W_V：

Token	Embedding	Key	Value
第 1 个 token	$[1,0]$	$[2,0]$	$[1,1]$
第 2 个 token	$[0,1]$	$[0,1]$	$[1,0]$
第 3 个 token	$[1,1]$	$[2,1]$	$[2,1]$

你看前两行：第 1 个 token 的 Key 还是 $[2,0]$，Value 还是 $[1,1]$。第 2 个 token 的 Key 还是 $[0,1]$，Value 还是 $[1,0]$。和上一步算出来的数字一模一样。

这不是巧合。$[1,0]$ 乘以 W_K 这个运算，不管你做一次还是做一百次，结果永远是 $[2,0]$。输入没变，矩阵没变，输出不可能变。第 1 个 token 的 K 和 V、第 2 个 token 的 K 和 V，这 4 次矩阵乘法全部是白算的。

只有第 3 个 token 的 K 和 V 是新的：$[1,1]\times W_K=[2,1]$，$[1,1]\times W_V=[2,1]$。这才是这一步里真正有用的计算。

接下来模型要算出第 3 个位置（最后一个位置）的注意力输出，这个输出将决定第 4 个 token 是什么。这里我们把完整过程走一遍，得到一个对照结果：等一下换成 KV Cache 版本，如果输出还是同一个向量，就说明缓存没有改变模型结果，只是省掉了重复计算。

先算第 3 个 token 的 Query 向量：

$$\begin{aligned} q_3 &= [1,1]\times W_Q \\ &= [1\times 1+1\times 1,\ 1\times 0+1\times 1] \\ &= [2,1] \end{aligned}$$

然后拿 q_3 去和每个 token 的 Key 做点积，得到注意力分数：

$$\begin{aligned} q_3\cdot k_1 &= [2,1]\cdot[2,0]=2\times 2+1\times 0=4 \\ q_3\cdot k_2 &= [2,1]\cdot[0,1]=2\times 0+1\times 1=1 \\ q_3\cdot k_3 &= [2,1]\cdot[2,1]=2\times 2+1\times 1=5 \end{aligned}$$

原始分数：$[4,1,5]$。

除以 $\sqrt{d_k}=\sqrt{2}\approx 1.41$：$[2.84,0.71,3.55]$。

Softmax 归一化：

$$e^{2.84}\approx 17.1,\qquad e^{0.71}\approx 2.0,\qquad e^{3.55}\approx 34.8$$

总和 ≈ 53.9。

注意力权重：$[17.1\div 53.9,\ 2.0\div 53.9,\ 34.8\div 53.9]\approx[0.32,0.04,0.64]$。

最后，用这组权重对所有 token 的 Value 做加权求和：

$$0.32\times[1,1]+0.04\times[1,0]+0.64\times[2,1]$$

逐个分量算：

- 第一维：$0.32\times 1+0.04\times 1+0.64\times 2=0.32+0.04+1.28=1.64$

- 第二维：$0.32 \times 1 + 0.04 \times 0 + 0.64 \times 1 = 0.32 + 0 + 0.64 = 0.96$

第 3 个位置的注意力输出：$[1.64, 0.96]$。

记住这个数字。等一下我们用缓存重新走一遍同样的步骤，你会看到结果完全相同。

20.1.3 把算过的存起来

既然前面的 K 和 V 不会变，那有一个再自然不过的想法：第一次算完之后，把结果存下来，后面直接取用。

现在换一种做法，重新走一遍刚才的两轮生成。第一轮该算的照样算，只是算完 K 和 V 以后顺手存下来；第二轮再需要前两个 token 的 K 和 V 时，不重新乘矩阵，直接从缓存里拿。

第一步，处理用户输入的 2 个 token。这一步和刚才没有任何区别：算出 2 个 token 的 K 和 V。唯一多做的事情是，算完之后把结果存到一个叫“缓存”的地方：

缓存内容	Key	Value
第 1 个 token	$[2, 0]$	$[1, 1]$
第 2 个 token	$[0, 1]$	$[1, 0]$

模型用这组 K 和 V 算出第 2 个位置的注意力输出，预测出第 3 个 token，Embedding 是 $[1, 1]$。到这里为止，和不用缓存的版本做的事情一模一样：第一步没有任何重复可以省。

第二步，生成第 4 个 token。现在差别来了。

不用缓存的版本，是把 3 个 token 的 K 和 V 全部重新算一遍。而用缓存的版本，只需要算第 3 个 token 一个 token 的 K 和 V：

$$k_3 = [1, 1] \times W_K = [2, 1]$$
$$v_3 = [1, 1] \times W_V = [2, 1]$$

然后把这组新的 K 和 V 追加到缓存里：

缓存内容	Key	Value
第 1 个 token	$[2, 0]$	$[1, 1]$
第 2 个 token	$[0, 1]$	$[1, 0]$
第 3 个 token（新追加）	$[2, 1]$	$[2, 1]$

第 1 个和第 2 个 token 的 K 和 V 没有重新计算，直接从缓存里取。

接下来算第 3 个位置的注意力输出。算法和不用缓存时完全一样：算 Query、打分、Softmax、加权求和。区别仅仅在于：k_1、k_2、v_1、v_2 是从缓存里取出来的，不是重新算的。

我们一步一步走，你对照着上一小节的数字看：

$$q_3 = [1, 1] \times W_Q = [2, 1]$$
$$q_3 \cdot k_1 = [2, 1] \cdot [2, 0] = 4$$
$$q_3 \cdot k_2 = [2, 1] \cdot [0, 1] = 1$$
$$q_3 \cdot k_3 = [2, 1] \cdot [2, 1] = 5$$

原始分数：$[4, 1, 5]$。和不用缓存时一样。

除以 $\sqrt{2} \approx 1.41$：$[2.84, 0.71, 3.55]$。和不用缓存时一样。

Softmax 之后的注意力权重：$[0.32, 0.04, 0.64]$。和不用缓存时一样。

对 Value 加权求和：

$$0.32 \times [1,1] + 0.04 \times [1,0] + 0.64 \times [2,1] = [1.64, 0.96]$$

第 3 个位置的注意力输出：$[1.64, 0.96]$。

和不用缓存时的结果一模一样。

这是理所当然的——比如第 1 个 token 的 K 和 V，k_1 不管是重新算出来的还是从缓存里取出来的，数字都是 $[2,0]$；v_1 不管是重新算的还是取出来的，数字都是 $[1,1]$。数字相同，后面所有的运算结果也相同。从缓存里取 $[2,0]$，和重新用 $[1,0]$ 乘以 W_K 算出 $[2,0]$，对后续的计算来说没有任何区别。

但代价的差距是巨大的。不用缓存时，这一步算了 3 个 token 的 K 和 V，共 6 次矩阵乘法。用缓存时，只算了 1 个 token 的 K 和 V，共 2 次。省掉的 4 次矩阵乘法得到的结果和缓存里存的数字一模一样，纯粹是在做无用功。

现在你可能注意到了一个事实：在这个过程中，前两个 token 的 Query 向量（q_1 和 q_2）也没有重新算。但我同样没有把它们放进缓存。为什么 Key 和 Value 要缓存，Query 不用？

从刚才的计算过程里就能看出来。生成第 4 个 token 时，我们只需要算第 3 个位置的注意力输出。算第 3 个位置的注意力输出，需要的是 q_3——第 3 个 token 的 Query。q_1 和 q_2 在上一步（生成第 3 个 token 时）就已经完成了它们的使命：q_1 用来算第 1 个位置的输出，q_2 用来算第 2 个位置的输出。这两个位置的输出已经算完了，后续步骤不会再用到它们的 Q。

而 K 和 V 不一样。k_1 和 v_1 在生成第 3 个 token 时被用了一次（q_2 和 k_1 做点积，v_1 参与加权求和），在生成第 4 个 token 时又被用了一次（q_3 和 k_1 做点积，v_1 参与加权求和），如果继续生成第 5 个、第 6 个 token，每一步都还要用。因为每个新 token 的 Query 都需要和所有历史 token 的 Key 做点积，对所有历史 token 的 Value 做加权求和。

Query 是“一次性消耗品”，用完一次就不再需要了。Key 和 Value 是“每次都要翻出来的档案”，每生成一个新 token 都要从头到尾翻一遍。

Query 不缓存，和下面要讲的因果掩码是两件事。它只和注意力计算的结构有关：第 i 个位置只需要自己的 Query 来算第 i 个位置的输出，后续步骤不会再拿这个 Query 出来用。因果掩码要解释的是另一个问题：旧 token 的 Key 和 Value 为什么存下来之后永远不用改。

所以最终的方案是：只缓存每一层每个 token 的 Key 和 Value，不缓存 Query。这就是“KV Cache”这个名字的由来。每来一个新 token，只算它的 Q、K、V，K 和 V 追加进缓存，Q 和缓存里所有 K 做点积算分数，对所有 V 做加权求和，得到输出。

20.1.4 为什么 K 和 V 一定不会变

刚才的例子里，我们用的是 1 层 Transformer、2 维向量。第 1 个 token 的 Key 无论算几次都是 $[2,0]$。但你可能还是有一个担心：真实的模型有几十层，会不会在某一层里，前面 token 的 K 或 V 因为新 token 的加入而发生变化？

不会。原因是因果掩码。

因果掩码的规则是：每个 token 只能看到自己和自己前面的 token，看不到后面的。第 1 个 token 只能看到自己，第 2 个 token 能看到第 1 个和自己，第 3 个能看到前三个。后面新来的 token，前面的 token 完全感知不到它的存在。

这里最容易想错的一点是：注意力机制确实会把不同位置的信息混合起来，但在 Decoder 里，这种混合是有方向的。第 3 个 token 可以看到第 1 个和第 2 个 token，把它们的信息混进自己的表示里；但第 1 个、第 2 个 token 看不到第 3 个 token，所以它们自己的表示不会因为第 3 个 token 来了就被重新改写。每一层都是这个规则：新位置可以读旧位置，旧位置不能读新位置。

再往后看，MLP 层也不会制造“未来 token 反过来影响过去 token”的通道。MLP 里的线性层是在每个 token 自己的向量内部做变换，混合的是同一个 token 的不同维度，而不是把不同 token 的位置混在一起算。LayerNorm、残差连接也是逐位置处理。也就是说，真正跨 token 读信息的是注意力；而在 Decoder 里，注意力又被因果掩码限制住了方向。

所以，当第 3 个 token 出现在序列末尾时，第 1 个 token 的注意力计算范围没有发生任何变化——它只看自己。第 2 个 token 的注意力计算范围也没有变化——它只看第 1 个和自己。它们在每一层的输入没变，注意力分布没变，输出也没变。输出没变，下一层的输入也没变。一层一层往上传，整个 Transformer 从第 1 层到最后一层，前面 token 的 K 和 V 都和之前算出来的完全相同——在数学上严格等价，一个 bit 都不差。

这也划定了 KV Cache 的使用边界：它只能用在有因果掩码的架构里，也就是 Decoder 架构。如果是没有因果掩码的 Encoder 架构，每个 token 能看到所有 token，新增一个 token 之后，所有位置的注意力分布都可能改变，K 和 V 就不能简单复用了。这是 Decoder-only 架构在推理效率上的又一个天然优势。

20.1.5 省了多少计算

回到我们的例子，把两种模式的计算量放在一起看。我们只数“算 K 和 V 的矩阵乘法次数”：每个 token 算一次 K 和一次 V，算 2 次。

生成哪个 token	需要处理几个 token	不用缓存的 K/V 乘法次数	用缓存的 K/V 乘法次数
生成第 3 个 token	2	$2 \times 2 = 4$	4（第一步没得省）
生成第 4 个 token	3	$3 \times 2 = 6$	$1 \times 2 = 2$（只算第 3 个 token）
生成第 5 个 token	4	$4 \times 2 = 8$	$1 \times 2 = 2$（只算第 4 个 token）
合计		18	8

第一行两列数字相同，是因为处理用户输入时，每个 token 的 K 和 V 都是第一次被算出来，没有旧结果可以复用，缓存还是空的。从第二行开始，缓存的威力就体现出来了。只生成了 3 个 token，K 和 V 的计算量就从 18 次降到了 8 次。

如果是生成 100 个 token 呢？假设用户输入了 2 个 token，模型需要生成 100 个新 token。

不用缓存的总计算量：$2 \times (2 + 3 + 4 + \cdots + 101) = 2 \times 5150 = 10300$ 次。

用缓存的总计算量：$4 + 2 \times 99 = 202$ 次（第一步处理 2 个输入 token 花 4 次，后续 99 步每步 2 次）。

10300 对 202，差了大约 50 倍。生成 1000 个 token，差距就是 500 倍。

如果只看 K/V 投影这块，不用缓存的计算量随序列长度的平方增长，因为每一步要处理的 token 数在递增，步数本身也在递增。用缓存后，K/V 投影的计算量随生成长度线性增长，每一步固定只算 1 个 token 的 K 和 V。

这就是为什么 KV Cache 不是一个可选的优化，而是所有推理引擎的标配。KV Cache 做的事情很直接：旧 token 的 K 和 V 算过一次就够了，后面不会变，就没有必要每一轮都重新算。因果掩码已经从数学上保证了旧 token 的 K 和 V 不可能变，重算它们就是在浪费算力和电费。

20.1.6 预填充和逐 token 解码

刚才的过程里，生成被自然地分成了两个阶段。第一个阶段是处理用户输入的那 2 个 token——这些 token 是一次性全部送进模型的，并行计算，算完之后把 K 和 V 存入缓存。这个阶段叫**预填充（Prefill）**。

第二个阶段是一个 token 一个 token 往外蹦——每一步只处理 1 个新 token，算它的 Q、K、V，K 和 V 追加进缓存，Q 和缓存里所有 K 做点积，对所有 V 加权求和，得到输出，预测下一个 token。这个阶段叫**逐 token 解码（Decode）**。

这两个阶段的名字本身不重要，但它们的性能特征截然不同，面试和实际工作中经常被问到。

Prefill 阶段把用户输入的所有 token 一口气并行送进去。如果用户的提问有 500 个 token，这 500 个 token 同时处理，GPU 的成百上千个计算核心可以同时开工，计算资源被充分利用。这个阶段的瓶颈是算力——矩阵乘法的量大，但 GPU 就是为这种大规模并行计算设计的，能跑满。

Decode 阶段完全是另一回事。每一步只处理 1 个 token，计算量很小——1 个 token 的 Q、K、V 矩阵乘法，加上 1 个 Q 和 n 个 K 的点积。GPU 有几千个计算核心，但每步只有这么点活，大部分核心闲着。这个阶段真正的瓶颈不是“算得快不快”，而是“读数据快不快”——每一步都要从显存里把缓存中所有历史 token 的 K 和 V 读出来送给计算核心。当序列很长的时候，这个“从显存搬数据”的时间反而比计算本身更先到达上限。

Prefill 还有一个你能直接感受到的影响：用户发送一条很长的消息之后，要等 Prefill 处理完才能看到模型开始输出第一个字。这段等待时间叫**首 token 延迟（Time to First Token）**。如果用户上传了一份两万字的文档让模型总结，Prefill 阶段要一次性处理几千个 token，可能需要好几秒。你在使用大模型产品时偶尔遇到的“发出问题后等了好一会儿才开始蹦字”，往往就是 Prefill 在处理长输入。

20.1.7 缓存要占多少显存

KV Cache 用空间换时间，省掉的是计算量，付出的是显存。缓存里存着每一层 Transformer 每个 token 的 Key 向量和 Value 向量，这些向量加起来要占多大的空间？

我们用一个 7B 参数量的模型来算一笔账。先按最基础的设定来：所有注意力头都有独立的 K 和 V，后面会讲怎么压缩。典型参数是：

- Transformer 层数：32 层
- 每个 token 的向量维度（d_{model}）：4096
- 精度：fp16，每个数字占 2 个字节

每个 token 在一层需要缓存 1 个 Key 向量和 1 个 Value 向量。1 个 Key 向量有 4096 个数字，每个数字 2 字节，就是 $4096 \times 2 = 8192$ 字节 $= 8$ KB。Value 同理，也是 8 KB。一层里一个 token 的 KV 缓存是 $8 + 8 = 16$ KB。

32 层加起来：32×16 KB $= 512$ KB，大约 0.5 MB。

也就是说，**每多缓存一个 token，要多占大约 0.5 MB 显存。**

这个数字看起来不大。但缓存里存的是序列中所有 token 的 KV：用户输入的加上模型生成的，每一个都要存。

上下文长度 4096 个 token 时：

$$512\ \text{KB} \times 4096 \approx 2\ \text{GB}$$

光是 KV Cache 就占了 2 个 G 的显存。而这个 7B 模型的权重本身在 fp16 下大约占 14 GB。

我们把不同上下文长度的情况列出来：

上下文长度	KV Cache 显存	和模型权重（14 GB）比
4K	2 GB	模型权重的 14%
32K	16 GB	超过了模型权重
128K	64 GB	模型权重的 4.6 倍

当上下文长度达到 32K 的时候，KV Cache 占的显存已经超过了模型权重本身。到 128K（很多新模型声称支持的上下文长度），一块 80GB 显存的 A100 显卡，模型权重 14 GB 加 KV Cache 64 GB 就是 78 GB，几乎把显存吃满。

而且这只是一个用户的开销。如果你部署的是一个在线服务，同时有 100 个用户在对话，每个用户的上下文都是 4K 个 token，那 KV Cache 的总显存是 100×2 GB $=$ 200 GB。你需要好几块显卡，光用来存缓存。

这就是为什么“最大上下文长度”不是随便设的：限制因素往往不在模型架构，关键在于 KV Cache 的显存会撑爆。后面讲显存估算的时候，KV Cache 会是推理场景下的重要一项。

20.1.8 用更少的 KV 头来压缩缓存

KV Cache 的显存和序列长度成正比、和层数成正比。这两个因素不太好动：缩短上下文会限制模型能力，减少层数等于换了一个模型。但还有一个维度可以做文章。

前面讲多头注意力的时候提到过，标准做法是每个头有独立的 W_Q、W_K、W_V，各自生成独立的 Q、K、V 向量。如果模型有 32 个注意力头，每一层每个 token 就有 32 份 K 和 32 份 V 需要缓存。

我们刚才算的 7B 模型，$d_{model} = 4096$，32 个头，每个头的维度是 $4096 \div 32 = 128$ 维。32 个头的 K 拼起来正好 4096 维。之前算出来的“每个 token 每层 16 KB”，就是 32 个头各自的 K 和 V 加在一起的大小。

GQA（Grouped Query Attention，分组查询注意力）的做法是：Query 照样用 32 个头，但 K 和 V 不用 32 个头，减少到比如 8 个头。每 4 个 Query 头共享同一组 K 和 V。名字里的 Grouped Query，说的就是 Query 头被分成了若干组，同组的 Query 共用一份 K 和 V。

具体来说，第 1 到第 4 个 Query 头各自有不同的 Query 向量，但它们点积时面对的是同一组 Key，加权求和时使用的也是同一组 Value。

Query 头数不变，模型仍然可以从 32 个不同角度去“提问”。但需要缓存的 K 和 V，从 32 份变成了 8 份。

对 KV Cache 的影响立竿见影。同样的 7B 模型设定：

原来 32 个 KV 头时，每层每个 token 缓存 $2 \times 4096 \times 2 = 16{,}384$ 字节 $= 16$ KB。

改成 8 个 KV 头后，每层每个 token 缓存 $2 \times (8 \times 128) \times 2 = 2 \times 1024 \times 2 = 4{,}096$ 字节 $= 4$ KB。

这个式子里的四个因子分别是：2 份（Key 和 Value 各一份）、8 个 KV 头、每个头 128 维、每个数字 2 字节。

32 层，4096 个 token：

- 原来：$16\ \text{KB} \times 32 \times 4096 = 2\ \text{GB}$
- 现在：$4\ \text{KB} \times 32 \times 4096 = 512\ \text{MB}$

KV Cache 缩小到原来的四分之一。

在 128K 上下文时，从 64 GB 降到 16 GB。从“一块 80 GB 的卡装不下”变成了“轻松装得下”。

这也是为什么很多近几年的主流模型会采用 GQA，比如 LLaMA 2 的 70B 版本、LLaMA 3、Qwen2、Mistral 等。压缩 KV Cache 是它很直接的工程动机之一。Query 的头数不变，注意力的表达能力损失有限，但推理阶段的显存节省是实打实的。

20.1.9 几个容易想错的地方

“KV Cache 在训练时也能用吧？”

不能。训练时，一句话的所有 token 是一口气并行送进去的，所有位置的 Q 都有用——因为每个位置都要产出预测值、计算损失、参与反向传播。整个 $n \times n$ 的注意力矩阵是一次性算完的，没有“先生成第 1 个再生成第 2 个”的过程，也就不存在重复计算。KV Cache 是纯粹的推理阶段优化。

“KV Cache 是一种近似加速，会损失精度。”

不会。这一节里你已经亲手验过了——缓存里的 $[2, 0]$ 和重新算出来的 $[2, 0]$ 是同一组数字，送进后续计算得到的结果完全相同。KV Cache 不像量化那样存在精度和速度之间的取舍。只有当你对 KV Cache 额外做量化压缩（比如从 fp16 压到 int8）时才会引入精度损失——但那是量化的事，不是 KV Cache 本身的事。

“序列越长，每生成一个 token 就越慢。”

这句话对了一半。有了 KV Cache 之后，每步的 K/V 投影计算量是固定的（只算 1 个新 token 的 Q、K、V）。但注意力打分那一步，也就是 1 个 Q 和缓存里所有 n 个 K 做点积，这个 n 确实随着生成过程线性增长。当序列很长的时候，真正的瓶颈往往不是“点积做了多少次乘法”，而是“要从显存里把那么大的 K 和 V 缓存读出来送给计算核心”。这时，数据搬运的速度先到上限了。

20.1.10 自检清单

1. KV Cache 缓存的是什么？为什么不缓存 Q？

   缓存的是每一层 Transformer 中每个 token 的 Key 向量和 Value 向量。不缓存 Q，是因为 Q 是一次性消耗品——第 i 个位置的 Q 只在计算第 i 个位置的输出时用一次，后续步骤再也不会用到。而 K 和 V 是每一步都要翻出来的，每生成一个新 token 都需要和所有历史 token 的 K 做点积、对所有历史 token 的 V 加权求和。

2. KV Cache 会损失精度吗？

   不会。缓存里的 K 和 V 和从头重算的结果是同一组数字。因果掩码保证了旧 token 的计算不受新 token 影响，所以从缓存里取出来的值和重新计算的值完全相同。

3. 为什么 KV Cache 显存会随着上下文长度和并发数线性增长？以 7B 模型、4K 上下文为例，大概是什么量级？

   因为每个历史 token 都要在每一层保存一份 K 和 V。上下文长度翻倍，单个请求要保存的历史 token 数翻倍，KV Cache 也跟着翻倍；并发请求数翻倍，总缓存也会翻倍。以 7B 模型、4K 上下文为例，单个请求的 KV Cache 大约是 2 GB。

4. Prefill 和 Decode 有什么区别？

   Prefill 是一次性并行处理用户输入的所有 token，计算量大但 GPU 利用率高。Decode 是一个 token 一个 token 往外蹦，每步计算量小但要频繁从显存读取整个 KV 缓存。两个阶段的性能瓶颈完全不同。

5. GQA 为什么能压缩 KV Cache？

   GQA 让多个 Query 头共享一组 K 和 V。比如 32 个 Query 头分 8 组，只需要 8 份 K 和 V，KV Cache 缩小到标准多头注意力的四分之一。Query 头数不变，注意力表达能力损失有限，但显存节省是实打实的。

20.2 模型量化：怎么把大模型塞进小显存

20.2.1 从一笔账算起

大模型的每一个参数，在计算机内部都是一个浮点数。浮点数有不同的格式：FP32 每个参数占 4 字节，BF16 每个参数占 2 字节。格式不同，同一个模型占用的存储空间就不同。如果这些细节有些模糊了，建议先翻回浮点数那一节快速复习一下，那里有完整的推导过程。这里你只需要记住一个结论：参数量 × 每个参数占的字节数 = 模型大小。

现在来算一笔账。一个 7B 模型有 70 亿个参数，用 BF16 存储，每个参数占 2 字节：

$$7{,}000{,}000{,}000 \times 2 = 14{,}000{,}000{,}000 \text{ 字节} \approx 14\text{GB}$$

14GB。如果你的显卡显存是 24GB，装得下，但剩余空间已经不多了，因为推理过程中还有别的东西要占显存。如果是一个 13B 模型，BF16 就要大约 26GB，24GB 的显卡直接装不下。

这时候你面前只有两条路：换一张更大的显卡，或者想办法把模型压小。

当然，最直接的办法是换一张更大的显卡。但显卡不是白来的，要花钱；如果你在公司里，采购什么卡、给不给你加机器，也大概率不是你一个人说了算。

所以更现实的方向，是想办法把模型压小一点。这里有一种被工业界广泛使用的方法，叫量化。

量化的核心想法极其朴素：既然每个参数用 BF16 要占 2 字节（16 位），那能不能用更少的位数来存？比如每个参数只用 8 位，甚至只用 4 位？

如果能做到，同一个 7B 模型：

- BF16：7B × 2 字节 = 14GB
- 8 位：7B × 1 字节 = 7GB
- 4 位：7B × 0.5 字节 = 3.5GB

从 14GB 压到 3.5GB，存储缩到四分之一。那个 13B 模型从 26GB 压到大约 6.5GB，24GB 的显卡不仅装得下，还绰绰有余。